

Benchmarking Key-Value Stores via Trace Replay

Edwin F. Boza, Cesar San-Lucas, Cristina L. Abad, Jose A. Viteri

Escuela Superior Politecnica del Litoral, ESPOL

{eboza,csanluca,cabad,javiteri}@fiec.espol.edu.ec

Abstract—Key-value stores are an important component of cloud applications. These NoSQL databases typically do not support ACID transactions; hence, their applications have diverged from traditional database workloads and are not well represented by benchmarks like TPC-C. In this context, YCSB emerged as the de facto benchmark for cloud serving stores, supporting a wide variety of synthetic workloads. However, no publicly available tool exists that can replay real traces against these systems; this is harmful to the community, as the replay of real workloads is an important system evaluation technique. As a result, others have developed ad-hoc closed-source replay tools that use undocumented replay models, thus making their results impossible to replicate. Furthermore, choosing and implementing the right replay model is not trivial. We propose a trace replay model suitable for key-value stores and describe KV-replay, an open-source replayer that implements this model. We show that using synthetic workloads leads to significant evaluation errors: As much as 33% error in miss rate for small cache sizes and 18% speedup overestimation. Evaluations show that KV-replay is accurate, fast and useful.

I. INTRODUCTION

Benchmarks that reproduce traditional database workloads, like TPC-C and TPC-H [1], are not representative of the workloads commonly served by cloud key-values stores [2]. To fill this gap, YCSB [2] emerged with a set of synthetic macro-benchmarks for cloud-serving stores.

While synthetic macro-benchmarks are important in systems evaluations, some system components are sensitive to dimensions not reproduced by synthetic generators. For example, temporal locality is an important dimension not reproduced by models like the Independent Reference Model (IRM) [3] used by most synthetic generators, including YCSB. Other dimensions not reproduced by YCSB include interarrivals (and burstiness) [4] and value sizes; these dimensions are easily included in trace replayers [5].

However (and to the best of our knowledge), there is no open source trace replayer for key-value stores. As a result, researchers end up implementing their own ad-hoc and undocumented replay tools. This is wasteful, and more importantly, prone to errors, as choosing the right way to replay a trace is not trivial [5]–[7].

Should we use a compilation-based or an event-based tool? Should the replay model use an open or closed loop [6]? Should we aim for accurate timings, or guarantee conservative or resource-oriented ordering of the events [8], [9]? How do we replay traces that describe heavy workloads that cannot be handled by a single replay client? How can we scale the workload? Different approaches can lead to different performance results [5]–[7]. The design and

implementation choices of ad-hoc replayers are rarely well documented, making it hard to reason about the results.

For example, many ad-hoc trace replay tools implement only the full speed timing policy. This policy is useful for stress testing, but (if used with an open replay model) does not consider any possible feedback between requests [7]. When a paper uses a replay tool to replay a trace at full speed, without indicating the replay model, it is impossible to assess the impact of the replay methodology on the results.

In addition, unlike traditional database systems for which there is a common API (ODBC), current key-value stores have their own proprietary APIs, making it hard to build a tool that can benchmark a wide variety of these products.

Our contributions are: (1) A trace replay model suitable for cloud key-value stores, and (2) an open-source tool whose design choices are well documented and leverages the YCSB framework, which has modules for 29 databases.

We show that KV-replay is as fast as YCSB, accurate in reproducing interarrivals and burstiness, and useful, as it lets us evaluate key-value stores under real workloads. Our tool can help others better assess their systems, as using synthetic workloads can lead to errors like overestimating the miss rate for small cache sizes (as much as 33%) and overestimating the speedup gained from increasing the cache size (18%).

II. MOTIVATION AND BACKGROUND

Key-value stores. Key-value stores are NoSQL databases that implement the associative array or dictionary abstract data type. Records are stored and retrieved using a unique key; the key has one or more values associated with it.

Due to their high scalability and performance, key-value stores are critical components of large Big Data and cloud systems. For this reason, they are a popular topic of study and significant efforts continuously go into improving their performance, scalability, reliability, cost, and power consumption [10]. For such efforts to be effective, we need proper evaluation methods, including micro- and macro-benchmarks, and tools for replaying real workloads.

YCSB. The Yahoo Cloud Serving Benchmark (YCSB) [2] is the de facto standard for benchmarking cloud databases and is widely used in industry and academia [4].

YCSB has six pre-configured synthetic workloads with read/write mixes inspired by common cloud use-cases. New workloads are created by configuring the key distribution (Uniform, Zipfian, Latest or Multinomial), the proportion of operations of each type, and the record sizes.

YCSB Limitations. YCSB does not model important workload dimensions, as discussed next. YCSB issues events at a constant rate (full speed); realistic *interarrivals* are necessary when burstiness affects performance [4], [11]. Similarly, a workload's *temporal locality* can impact a system's performance. For example, it is well known that caches (a common use of key-value stores) are affected by temporal locality [12]–[15], and should not be evaluated using only the Independent Reference Model (IRM), in which every access to an object is independent of prior ones [13], [14]. Nonetheless, YCSB uses this model¹. *Value size* can also affect performance; for example, Memcached has slabs for different size ranges. If all values have the same size (as with YCSB), then contention on the corresponding slab increases.

By adding support for trace replay to YCSB, we address these issues, as our trace format contains these dimensions.

III. MODEL

Replayers can be **compilation-based or event-based** [7]. Compilation-based replayers create a program that emulates the activity captured in the trace; for example, Berkley's SWIM for Hadoop [16]. Event-based replayers are generic programs that can replay the events captured in a trace; they are frequently used in storage systems [5], [7]–[9], [17]–[19]. The event-based approach is the most suitable for key-value stores, as the goal is to support traces from different applications while compilation-based replayers are better suited for single application benchmarks.

Workload generators can be **open or closed** [6], [7]. In the open loop model, events are independent from each other. In a closed loop generator, one or more clients/threads issue events one after the other, with some think time in between; an event is issued only after the prior one completes. In a partly-open model, clients arrive at the system at different times and issue requests during a *session*; events in a session are issued in a closed loop. For the case of replayers, the closed and partly-open loops are equivalent: The difference is the arrival of clients to the system and this is given by the trace and not by a stochastic arrival process.

We assume that the delay between two requests from a client comes from the response time plus the client think time, a common assumption in trace replay. An alternative is to ignore interarrivals and simulate think-time [6].

Database benchmarks like TPC-C and TPC-H, use a compilation-based, closed model [6]. For RDBMSs, it makes sense to use a compilation-based approach and issue complex SQL queries instead of issuing events that access single records. Queries are issued in closed loop to model the dependencies in a transaction or a query pipeline.

¹YCSB has one workload that does not use the IRM, called Zipfian Latest. However, it provides a non-realistic way to simulate temporal locality and was rarely used in the papers we surveyed.

Key-value stores provide weaker consistency models than ACID databases, and are used for applications outside traditional Online Transaction Processing (OLTP) or decision support systems. Examples of applications supported by cloud key-value stores include [2], [10], [20], [21]:

- **Caching objects**; e.g., a user profile cache, where profiles are constructed in another system like Hadoop.
- **Photo tagging**, where adding a tag is an update; most operations are to read tags.
- **User database**, where users read and modify their records or the system records user activity.
- **Tracking counters**; e.g., user engagement counters, like tracking clicks to objects.
- **Session stores**, recording actions in user sessions.
- **User status updates** in social networks; users update their own status and read the latest updates of others.
- **Publish-subscribe channels**; e.g., stock price tracking, where subscribers of a channel can track a stock price.

Based on the dynamics of these workloads, we argue that—unless workload analysis suggests otherwise—an open loop should be used to benchmark cloud key-value stores. Intuitively, the open model is well suited for many cloud applications; either because it captures well the actual behavior of the system (each event is independent of the others) or because even though there are a few dependencies (mainly in accesses from a client session), the number of events per session is small and the time between dependent operations is dominated by user think time and not by system response time. For example, a user tagging her pictures will do so by browsing them and manually adding tags as needed.

Consider the YouTube trace used in Section V: Only 0.13% and 0.004% of the events from a single client have an interarrival time smaller than the maximum and average key-value store latency observed in our tests, respectively. For this workload, using a closed loop model (in which the events coming from the same client are replayed only after prior events from that client have finished) is indistinguishable from replaying the trace using an open model.

Next, we describe three specific architectures and discuss which model is appropriate for replaying their workloads.

Cloud architecture 1: Frequently, a microservices tier sits between the end-user and the key-value store. The microservice processes run on application servers like node.js which use async (non-blocking) calls to issue requests to the database layer. A trace recorded in this setting could naively suggest a closed loop (few clients, each issuing many requests) when in fact these requests are independent of each other. We suggest using an open loop to replay the trace.

Cloud architecture 2: For end-user-facing cloud stores like Tarantool (tarantool.org), which combines an application server with a database and can be queried directly by web clients, one can follow the guidelines provided by Schroeder et al. [6] to choose the right replay model: Identify per-client sessions and count the number of requests in each session.

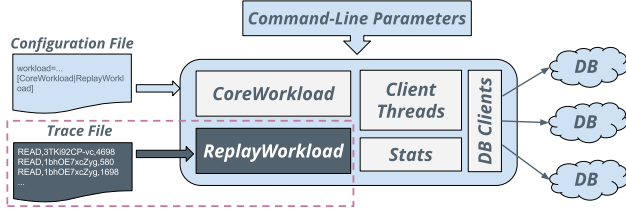


Figure 1. KV-replay's architecture, as an extension to YCSB.

A partly-open system behaves similarly to an open system when the expected number of requests per session is small (≤ 5 [6]) so it is safe to choose an open model. For longer sessions, requests should be issued in a partly-open loop.

Cloud architecture 3: In large key-value store farms, it makes sense to use a fast and lightweight proxy like Twemproxy (github.com/twitter/twemproxy), to pipeline client requests and reduce the connection pressure at the database. The right model would depend on how the trace was recorded. If the trace was recorded at the key-value store layer, then a closed loop represents the system behavior. If the trace was recorded at the proxy tier, then one should follow the guidelines for cloud architecture 2.

In short, different workloads call for different replay models. Naive trace replay implementations tend to use a closed approach [6] as it is easier to implement than the open model. We recommend using the open model by default, but our tool also supports the closed loop model for advanced users that believe their workloads call for it.

Next, we describe how we implemented these models and other design choices like the ordering and timing policies.

IV. DESIGN AND IMPLEMENTATION

We implemented KV-replay as an extension to YCSB, allowing us to benefit from its client plug-ins (as of 1/2017, YCSB has clients for 29 cloud stores plus a ODBC client).

To implement the open model we used Java's ScheduledExecutorService, which lets us issue timed events. Simply sleeping between events does not work when two events (A and B) occur so close to each other that B must be issued before A's response is received (YCSB uses blocking calls). Our evaluations show that this approach can reproduce the timings in a trace with millisecond accuracy.

Figure 1 shows the architecture of KV-replay. Unlike the original CoreWorkload module, which issues synthetic, random workloads, KV-replay's ReplayWorkload module replays the events recorded in a trace provided by the user.

To achieve a high throughput, the user can use multiple clients, each with one or more threads. The clients use a time-based barrier to start issuing events at the same time (accuracy discussion in Section V).

Benchmarking a key-value store with KV-replay requires running the following phases: An (optional) data loading phase and a workload execution phase. In the first phase, we prepare the database by populating it with keys, so that those

Table I
REPLAY MODELS AND POLICIES SUPPORTED BY KV-REPLAY.

Replay mode	Ordering guarantees	Timing policies
Open	None	Original and scaled
Closed	Conservative [8]	Full, original, scaled
Closed/Partly-open	Resource-oriented [9]	Original and scaled

keys can be accessed during the second phase, in which the trace is replayed. We recommend that users run KV-replay as a real-time process to avoid its preemption.

Input trace format. Records contain the following fields <Command, Object ID, Timestamp, Size>. The timestamp is optional because it is not needed when replaying a trace at full speed. The size is optional because the user may choose to use the same (configurable) size for all values. The value used for an insert or update is randomly generated by YCSB.

Missing events. Trace capture methods may miss a small percentage of events [5], [8]. This may be a problem when a missing event affects later operations in the trace.

KV-replay assumes that any access (*read*, *update* or *delete*) before an *insert* of the corresponding key indicates that the key had been inserted in the database before recording the trace. For this reason, there is a *preparation* phase where keys are loaded in the database.

However, two other explanations for a missing event exist: (a) The operation uses a key that does not exist and should fail, or (b) the insertion occurred during the capture period but the recording method failed to capture it.

Case (a) can be inferred from the trace if it contains return values (SUCCESS or FAILURE); if that's the case, the user can skip the preparation phase and do a manual preparation using a script that comes with KV-replay.

For case (b), the user can do the manual preparation phase and add the missing insertion operations wherever she thinks is appropriate. Alternatively, we provide a pre-processing script that adds *inserts* to keys that need them, *X* milliseconds prior to the first access to the key.

Replay models and ordering guarantees. KV-replay supports three replay models (summarized in Table I):

1) *Open (default)*: Requests are issued independently of each other. May require multiple clients for heavy traces (i.e., if the events must be replayed faster than the maximum throughput supported by a single client).

2) *Closed with conservative ordering*: Operations are issued only after all prior operations have completed [8]; only supported in single-thread, single-client mode, as these are not synchronized (other than a barrier-based synchronized start) and the conservative ordering can be violated if multiple threads or clients are used.

3) *Closed with resource-oriented ordering*: Requests from a client are guaranteed to be executed in the order in which they were recorded [9], and after prior requests from that client have completed; the same as a **partly-open loop**.

Timing policies. KV-replay supports three timing policies that determine how fast to replay the events.

1) *Full speed*: Events are issued one after the other, but only after the prior operation has finished as the operations are invoked using blocking calls; this is the same approach used by YCSB. Only applicable to the closed model.

2) *Original speed*: The trace interarrivals are mimicked. If used with a closed loop, events that are too close to each other may not be replayed at the original speed; two consecutive events with an interarrival time shorter than the database latency can't be issued on time using blocking calls.

3) *Scaled speed*: A speedup factor, f , can scale the replay speed up or down. Gradually increasing the speed is the recommended way of stress testing with the open model.

Workload scaling. When replaying a trace in a system smaller or larger than the original, one may want to scale the workload up or down, temporally, spatially, or both [5]².

We support **temporal scaling** via a scaling factor, allowing us to increase the pressure on the system or to replay a trace on a system smaller or slower than the original.

KV-replay supports three **spatial scaling** techniques: Cloning [8], sub-tracing [8] and intensifying [22]. *Cloning* scales up the number of operations and objects; each cloned trace is replayed against a separate set of keys. *Sub-tracing* scales down a trace spatially by decomposing it into multiple sub-traces, and issuing the workload of a subset of those sub-traces. *Intensifying* scales up a trace by sub-tracing it and then aligning the sub-traces at time zero. The keys in a sub-trace are prepended with the sub-trace ID so that each sub-trace accesses an independent set of keys. The merged sub-traces are replayed as a heavier but shorter workload.

Limitations and discussion. KV-replay achieves millisecond accuracy in the event timings. It is hard for benchmarking tools operating on non-real-time operating systems to achieve accuracy in the microsecond range [23]; reasons include: scheduling granularity and unpredictable performance due to interrupts, locking, resource contention, and kernel complexities. Anderson et al. solved the problem of achieving microsecond accuracy on a non-real-time OS through keeping time by spinning and other tricks detailed in [23]. In the future, these techniques can be adopted in KV-replay. However, our tests show that these errors are only for the per-event expected vs. achieved timing; other time dimensions, like interarrival time and burstiness are accurately reproduced (see Section V).

Finally, we need publicly available key-value store traces. Until such traces become available, other traces can be used, like those from YouTube video requests [24], Wikimedia downloads [25], and Yahoo HDFS requests [26].

²Scaling modifies the workload and may affect results. If a researcher is inclined to scale a workload, our tool makes it possible to do it reproducibly.

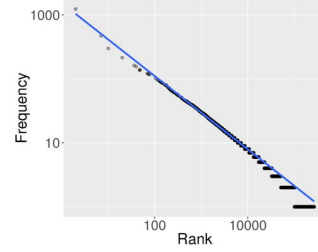


Figure 2. Rank-frequency distribution of the video popularity in the YouTube trace. The line shows the best fit to the Zipf distribution.

V. EXPERIMENTAL EVALUATION

We evaluated KV-replay's accuracy, usefulness and performance. We show averages across five runs. The experiments that compare the results of YCSB vs. KV-replay use a closed loop at full speed so that the comparison is fair, as this is the model used by YCSB. This section presents a subset of the results of our experiments; additional results can be found in an online appendix (see our Github repository).

Experimental setup. We ran the experiments on virtual machines running CentOS v7.2.1511 x86_64 with 1GB RAM and 1 vCPU, on VirtualBox v5.1.2. Our servers had the same version of CentOS, 4GB of RAM, Intel Core2 Quad CPU Q6600 @ 2.40GHz. The versions of the databases were: Redis 2.8.19, Redis Cluster 3.2.3, Memcached 1.4.15, Riak KV 2.1.4 and Hazelcast 3.6.4.

Workload. We used a trace of YouTube requests [24] (trace T5; 16K clients, 300K clips). Videos are served by CDNs, but the workload is strongly correlated with a common use-case of key-value stores: Content engagement counters (views, likes, dislikes), loaded in tandem with the page.

We used this workload in two ways: (1) We replayed the trace, and (2) found the best fit of the popularity distribution of the YouTube objects to a Zipf distribution (coefficient = 0.5, Figure 2), and configured YCSB with it. To facilitate result comparison, the key and value sizes were the same in all the experiments (23 and 1K bytes, respectively).

YCSB supports other popularity distributions in addition to Zipf. We ran experiments using the Zipf, Uniform and Latest distributions, but report only the Zipf results because it was the one that best approximated the real trace results (smallest Root Mean Squared Errors, RMSE), thus making it a fairer comparison. In addition, YCSB's Zipf was the distribution most commonly used in the papers we surveyed.

Key-value stores. We ran experiments for the top four cloud key-value stores [27]: Redis, Memcached, Riak KV (based on DynamoDB's architecture), and Hazelcast, with their default configurations³. We disabled persistency, for those databases that support it, because we wanted to study them

³Except for Riak KV, where we used 8 instead of 64 partitions to get a smooth miss rate curve like the one of the other databases.

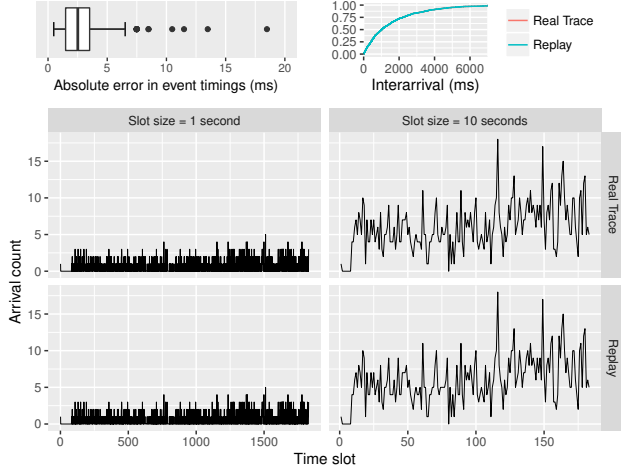


Figure 3. Accuracy of KV-replay. Top left: Box plot of the absolute timing errors in the replayed workload. Top right: CDFs of the interarrivals for the real trace and the replayed workload. Bottom: Burstiness in the arrivals.

when acting as in-memory stores. Caching is a common use of in-memory stores, so we configured the databases to use LRU eviction, for varying cache sizes.

Accuracy. We ran tests using a single KV-replay client and a distributed test with the workload divided between ten clients. We used the open loop model with the “original” timing policy. The goal was to see if we could reproduce three timing-related metrics: Event timings, interarrivals, and arrival burstiness. We report the results of the distributed test; the single client results were slightly better.

Figure 3 shows the accuracy of KV-replay. We are able to reproduce the per-event timings recorded in the trace with a small error (median = 2.04ms). The cumulative distribution functions (CDFs) of the interarrivals for the real trace vs. the replayed workload are almost identical, with a Kolmogorov-Smirnov distance between the two curves of 0.0045 (i.e., an error smaller than 0.46 percentage points).

The burstiness is also accurately reproduced; we quantify this by calculating the Hurst exponent of the arrivals time series using R/S analysis and a window of 1K milliseconds (0.7823 vs. 0.7818, for the real trace and the replay) and 10K milliseconds (0.8564 vs. 0.8565). The Hurst parameter is a measure of the burstiness of a time series (in this case, the counting process of the arrivals) [28].

Note that part of the timing errors are due to clock synchronization between the virtual machines. When we ran all the ten clients on a single machine, the timings were more accurate: Median of 1.36ms instead of 2.04ms, and 90% percentile of 3.6ms instead of 5.5ms).

Usefulness. The goal was to see how a workload dimension not reproduced by YCSB affects results: Realistic temporal locality. We ran the experiments using YCSB (Zipf) and KV-replay, both accessing the same number of keys.

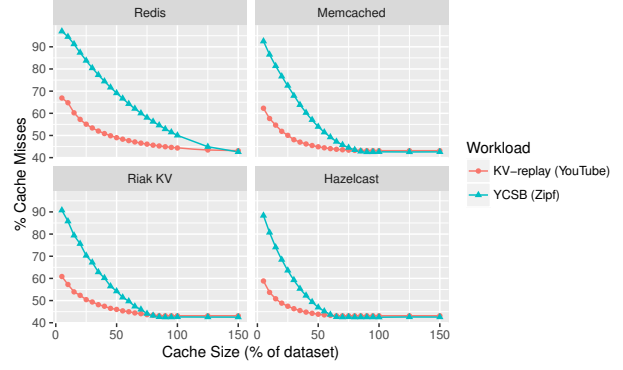


Figure 4. MRCs for in-memory DBs acting as caches with LRU eviction, for two workloads: Zipf (YCSB) and a real trace (KV-replay).

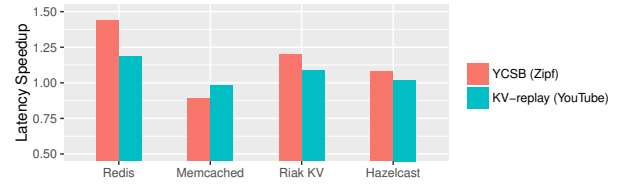


Figure 5. Speedup (in read latency) obtained from using a large cache that can hold all the data vs. a cache that can hold 10% of the data.

Figure 4 shows that the errors between the miss rate curves (MRCs) of the synthetic vs. the real workload can be as large as 30%. The synthetic workload cannot approximate the real results due to the lack of temporal locality⁴. As a result, a cache can be incorrectly provisioned having a negative impact in cost and performance.

Figure 5 shows the speedup (in read latency) from using a cache that can hold all the data vs. a small cache that can hold 10% of the data. The results show that the synthetic benchmark overestimates the speedup by as much as 18%.

Figure 6 compares the MRCs of the eviction policies of different databases as they rarely implement pure LRU but rather some approximation of it (e.g., to reduce locking contention). The synthetic benchmark considerably overestimates the differences between the MRCs.

Record size can also affect results. To test this we used Memcached, which has *slabs* for different size ranges. If all values have the same size, contention on the corresponding slab increases, leading to increased latency and reduced throughput, as shown in Figure 7. We configured Memcached with varying cache sizes and obtained similar results in all cases; the figure shows the results of the runs with a 1.5GB cache. To generate the workload, we used KV-replay with the YouTube trace in two ways: With a fixed size value size (same as vanilla YCSB) and with variable size records. For the latter, the value sizes were chosen so that the records

⁴YCSB's Latest performed even worse because it simulates a workload with a temporal locality significantly higher than the YouTube trace.

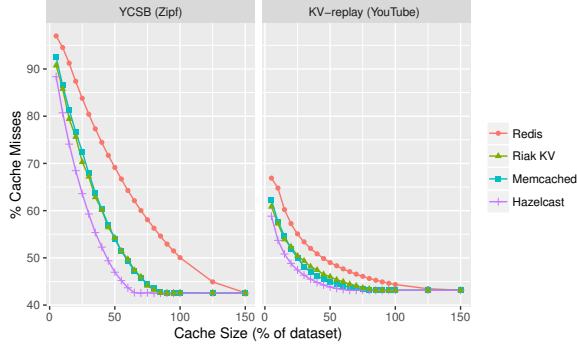


Figure 6. MRCs when using a synthetic benchmark and the real workload.

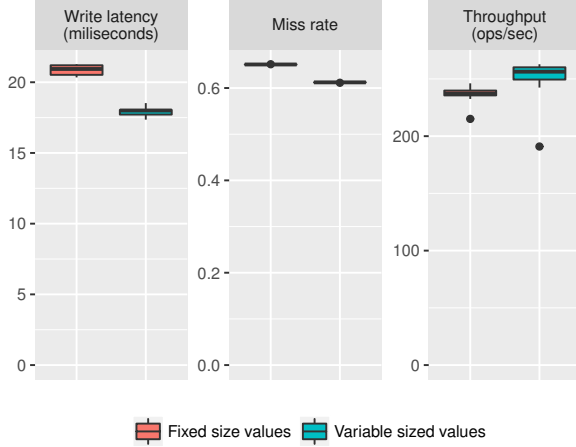


Figure 7. Inserting fixed size vs. variable sized records on Memcached.

were uniformly distributed across all Memcached slabs; for the former, the value size was the same as the mean size of the values in the variable-sized experiment (93862 bytes).

We now consider a load balancing experiment on a Redis Cluster with 10 nodes. Redis Cluster divides the key space between multiple nodes. We ran experiments at full speed and counted the number of accesses per node for each 1-second slot. Figure 8 shows the box plot of the access counts; the differences in range ($max - min$) and interquartile range ($IQR = Q3 - Q1$) show that the results for the synthetic workload are more dispersed. Due to the temporal locality in the YouTube workload, accesses to a key tend to be close in time creating hotspots, whereas in the Zipfian workload they are randomly distributed throughout the duration of the experiment (some hotspots appear, but due to keys with high popularity, not to temporal locality).

Performance. Our goal was to see if KV-replay could issue events as fast as YCSB. For KV-replay, we used the closed model with multiple threads, which may violate conservative ordering [8]. This experiment assumes that the events are independent but replayed using the closed model because this model throttles the throughput by using blocking calls. We did not use other models because they are not supported by YCSB and the results would not be comparable.

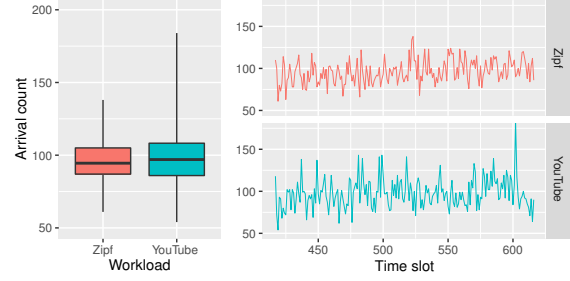


Figure 8. Access hotspots in a 10-node Redis Cluster. Left: Box plot of number of requests each node received in each 1-second slot, aggregated for all nodes. Right: A sample of the arrivals, for one node in each experiment.

In our results, shown in our appendix at our Github repo, we observe no significant difference in throughput; KV-replay can stress the system as effectively as YCSB.

VI. RELATED WORK

A rich body of work exists in the area of trace replay for benchmarking enterprise storage systems [5], [7]–[9], [17]–[19], web servers [6], [29], etc. Our work builds up on these studies by selecting the models and policies that are applicable to cloud serving key-value store workloads.

A few benchmarking tools with trace replay functionality have been proposed for cloud systems [16], [30], [31]. SWIM [16] is a compilation-based, open-loop replayer for Hadoop; it parses Hadoop logs to create a MapReduce workload that reproduces the input-output and arrival characteristics of the original workload.

iGen [30] is a benchmark for cloud file storage systems; it supports trace replay using an open model.

OLTP-Bench [31] is a benchmark for traditional DBMSs, including relational cloud databases like Amazon’s RDS and Microsoft’s Azure SQL; NoSQL databases are not supported. OLTP-Bench supports the replay of some pre-defined traces; however, it does not include non-transactional workloads. It uses a closed model with per-transaction ordering guarantees.

To the best of our knowledge, we are the first to propose a trace replay model and benchmarking tool for cloud serving key-value stores. KV-replay supports multiple replay models, covering a variety of cloud use cases.

VII. CONCLUSIONS

We described a model for replaying traces to benchmark cloud key-value stores. We have implemented this model in an open-source tool called KV-replay, available for download at <https://github.com/ebozag/KV-replay>.

Acknowledgments. This work was partially funded by a Google Research Award. We thank our anonymous reviewers for their valuable comments which helped improve the paper.

REFERENCES

- [1] M. Poess and C. Floyd, "New TPC benchmarks for decision support and web commerce," *SIGMOD Rec.*, vol. 29, no. 4, 2000.
- [2] B. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2010.
- [3] S. Vanichpun and A. M. Makowski, "The output of a cache under the Independent Reference Model: Where did the locality of reference go?" *SIGMETRICS Performance Evaluation Review*, vol. 32, no. 1, 2004.
- [4] R. Pitchumani, S. Frank, and E. L. Miller, "Realistic request arrival generation in storage benchmarks," in *Proceedings of the IEEE Symposium on Mass Storage Systems and Technologies (MSST)*, 2015.
- [5] A. Traeger, E. Zadok, N. Joukov, and C. Wright, "A nine year study of file system and storage benchmarking," *ACM Transactions on Storage (TOS)*, vol. 4, no. 2, 2008.
- [6] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proceedings of the USENIX Conference on Networked Systems Design & Implementation (NSDI)*, 2006.
- [7] T. E. Pereira, F. Brasileiro, and L. Sampaio, "File system trace replay methods through the lens of metrology," in *Proceedings of the IEEE Symposium on Mass Storage Systems and Technologies (MSST)*, 2016.
- [8] N. Zhu, J. Chen, and T. Chiueh, "TBBT: Scalable and accurate trace replay for file server evaluation," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2005.
- [9] Z. Weiss, T. Harter, A. Arpaci-Dusseau, and R. Arpaci-Dusseau, "ROOT: Replaying multithreaded traces with resource-oriented ordering," in *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*, 2013.
- [10] B. Atikoglu, Y. Xu, E. Frachtenberg, S. Jiang, and M. Paleczny, "Workload analysis of a large-scale key-value store," in *ACM SIGMETRICS Performance Evaluation Review*, vol. 40, no. 1, 2012.
- [11] A. Al-Shishtawy and V. Vlassov, "Elastman: Elasticity manager for elastic key-value stores in the cloud," in *ACM Cloud and Autonomic Computing Conference (CAC)*, 2013.
- [12] S. Vanichpun and A. M. Makowski, "Comparing strength of locality of reference – Popularity, majorization, and some folk theorems," in *Proceedings of the IEEE International Conference on Computing Communications (INFOCOM)*, 2004.
- [13] C. Abad, M. Yuan, C. Cai, Y. Lu, N. Roberts, and R. Campbell, "Generating request streams on Big Data using clustered renewal processes," *Performance Evaluation*, vol. 70, no. 10, 2013.
- [14] S. Traverso, M. Ahmed, M. Garetto, P. Giaccone, E. Leonardi, and S. Niccolini, "Temporal locality in today's content caching: Why it matters and how to model it," *SIGCOMM Computer Communication Review*, vol. 43, no. 5, 2013.
- [15] G. Bianchi, A. Deti, A. Caponi, and N. Blefari Melazzi, "Check before storing: What is the performance price of content integrity verification in LRU caching?" *SIGCOMM Computer Communication Review*, vol. 43, no. 3, 2013.
- [16] Y. Chen, A. Ganapathi, R. Griffith, and R. Katz, "The case for evaluating MapReduce performance using workload suites," in *Proceedings of the IEEE International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS)*, 2011.
- [17] N. Joukov, T. Wong, and E. Zadok, "Accurate and efficient replaying of file system traces," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2005.
- [18] M. P. Mesnier, M. Wachs, R. R. Sambasivan, J. Lopez, J. Hendricks, G. R. Ganger, and D. O'Hallaron, "TRACE: Parallel trace replay with approximate causal events," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2007.
- [19] V. Tarasov, S. Kumar, J. Ma, D. Hildebrand, A. Povzner, G. Kuenning, and E. Zadok, "Extracting flexible, replayable models from large block traces," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2012.
- [20] M. Gravell, "Blogpost: async redis await booksleeve [online]," <http://blog.marcgravell.com/2011/04/async-redis-await-booksleeve.html>, Apr. 2011, [Accessed: 25- July- 2016].
- [21] "Stackoverflow question: pub/sub middleware with persistent storage [online]," <http://stackoverflow.com/questions/19117153>, Oct. 2013, [Accessed: 8- Aug- 2016].
- [22] Y. Hua, H. Jiang, Y. Zhu, D. Feng, and L. Xu, "SANE: Semantic-aware namespace in ultra-large-scale file systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 5, 2014.
- [23] E. Anderson, M. Kallahalla, M. Uysal, and R. Swaminathan, "Buttress: A toolkit for flexible and high fidelity I/O benchmarking," in *Proceedings of the USENIX Conference on File and Storage Technologies (FAST)*, 2004.
- [24] M. Zink, K. Suh, Y. Gu, and J. Kurose, "Characteristics of YouTube network traffic at a campus network - measurements, models, and implications," *Computer Networks*, vol. 53, no. 4, 2009.
- [25] G. Urdaneta, G. Pierre, and M. van Steen, "Wikipedia workload analysis for decentralized hosting," *Computer Networks*, vol. 53, no. 11, 2009.
- [26] Yahoo Webscope, "Yahoo Hadoop grid logs, version 1.0," <https://webscope.sandbox.yahoo.com>, [Accessed: 12- Sep- 2016].
- [27] "DB-engines ranking of key-value stores [online]," <http://db-engines.com/en/ranking/key-value+store>, Jul. 2016, [Accessed: 18- July- 2016].
- [28] M. E. Crovella and A. Bestavros, "Self-similarity in world wide web traffic: Evidence and possible causes," *IEEE/ACM Transactions on Networking*, vol. 5, no. 6, 1997.
- [29] M. Rawat and A. Kshemkalyani, "SWIFT: Scheduling in web servers for fast response time," in *Proceedings of the IEEE International Symposium on Network Computing and Applications (NCA)*, 2003.
- [30] Z. Ren, B. Xu, W. Shi, Y. Ren, F. Cao, J. Lin, and Z. Ye, "iGen: A realistic request generator for cloud file systems benchmarking," in *Proceedings of the IEEE International Conference on Cloud Computing (CLOUD)*, 2016.
- [31] D. E. Difallah, A. Pavlo, C. Curino, and P. Cudre-Mauroux, "OLTP-Bench: An extensible testbed for benchmarking relational databases," *Proceedings of the VLDB Endowment (PVLDB)*, vol. 7, no. 4, Dec. 2013.